# Vector（向量）

Leo Shiang，2022/11/05


- Vector（向量）
  - Vector 為陣列的擴充
  - 引用套件
  - constructor
    - 指定長度
    - 傳入多個數字作為初始值
    - 傳入陣列當作初始值
  - add
    - 說明
    - 參數
    - 回傳
    - 例外
    - 範例
      - 回傳的結果是新向量
      - 與 Vector 相加
      - 與數字相加
      - 與其他類型的資料相加
  - append
    - 說明
    - 參數
    - 回傳
    - 範例
      - 空向量附加一個數值
      - 加入一個有三個元素的陣列
  - clone
    - 說明
    - 參數
    - 回傳
    - 範例
  - cross
    - 說明
    - 參數
    - 回傳
    - 例外
    - 範例
      - $\vec{a} \times \vec{a} = 0$
      - $\vec{a} \times 0 = 0$
      - 反交換律 $\vec{a} \times \vec{b} = -\vec{b} \times \vec{a}$
      - 加法的左分配律 $\vec{a} \times (\vec{b} + \vec{c}) = \vec{a} \times \vec{b} + \vec{a} \times \vec{c}$

加法的右分配律 $(\vec{a}+\vec{b})\times\vec{c}=\vec{a}\times\vec{c}+\vec{b}\times\vec{c}$
$(\lambda\vec{a})\times\vec{b}=\lambda(\vec{a}\times\vec{b})=\vec{a}\times(\lambda\vec{b})$
$|\vec{a}\times\vec{b}|=|\vec{b}\times\vec{a}|$
$|\vec{a}\times\vec{b}|^2=|\vec{a}|^2|\vec{b}|^2-(\vec{a}\cdot\vec{b})^2$

dot

說明

參數
回傳
例外
範例

交換律 $\vec{a}\cdot\vec{b}=\vec{b}\cdot\vec{a}$
分配律 $\vec{a}\cdot(\vec{b}+\vec{c})=\vec{a}\cdot\vec{b}+\vec{a}\cdot\vec{c}$
內積是雙線性算子 $\vec{a}\cdot(r\vec{b}+\vec{c})=r(\vec{a}\cdot\vec{b})+(\vec{a}\cdot\vec{c})$
乘以純量時滿足 $(c_1\vec{a})\cdot(c_2\vec{b})=(c_1c_2)(\vec{a}\cdot\vec{b})$
兩個非零向量 $\vec{a}$ 和 $\vec{b}$ 是正交的， $\vec{a}\cdot\vec{b}$ 必為 0

## Vector 為陣列的擴充

你可以將 Vector 視為一個加上擴充功能的陣列。

> 用 OOP 的說法：Vector 繼承了 Array。

在下列的程式中我們建立了一個 Vector 並設定長度為 3，接著用陣列的寫法去設定元素編號 0,1,2 的值，最後呼叫陣列的 reduce 方法加總所有元素。

```
const v = new Vector(3)
v[0] = 1
v[1] = 2
v[2] = 3
const sum = v.reduce((prev, curr) => prev + curr)
console.log(sum) // 6
```

> Vector和陣列的元素編號都是從 0 開始。

如果用 Vector 的 sum 函數也是可以計算所有元素的加總，只是程式看起來比較簡潔。

```
const v = new Vector(3)
v[0] = 1
v[1] = 2
v[2] = 3
console.log(v.sum()) // 6
```

和陣列一樣，可以用 `for` 迴圈遍歷 Vector 的每一個元素

```
const v = new Vector(3)
v[0] = 1
v[1] = 2
v[2] = 3
for (let i = 0; i < v.length; i++) {
  console.log(v[i])
}
```

或是用陣列的 `forEach` 方法：

```
const v = new Vector(3)
v[0] = 1
v[1] = 2
v[2] = 3
v.forEach(x => console.log(x))
```

從以上的例子可以得知 Vector 除了具備了陣列的所有特性之外，還添加了新的功能（在本文後面會介紹）。

# 引用套件

要使用 Vector ，你必須先安裝套件

```
npm i dolphin-node-core
```

在 javascript 程式的開頭引用套件 `dolphin-node-core`

```
const Vector = require('dolphin-node-core')
```

## constructor

Vector 是使用 ES6 建立的 class（類別），因此要用 constructor（建構子）來建立 Vector。

```
const v = new Vector()
```

如果呼叫 constructor 時沒傳入任何參數，Vector 的 length 屬性會等於 0。

```
const v = new Vector()
console.log(v.length) // 0
```

### 指定長度

如果呼叫 constructor 時傳入大於零的數字，該數字代表此 Vector 的長度，每一個元素會被賦予初始值 0。

```
const Vector = require('dolphin-node-core')
const v = new Vector(3) // 此 Vector 有 3 個元素
console.log(v.length) // 長度等於 3
consloe.log(v) // [0, 0, 0]
```

如果長度小於零會產生錯誤

```
const Vector = require('dolphin-node-core')
const v = new Vector(-1) // ParamError: 長度必須大於等於零
```

### 傳入多個數字作為初始值

```
const Vector = require('dolphin-node-core')
const v = new Vector(-1, -2, -3) // 此 Vector 有 3 個元素
console.log(v.length) // 長度等於 3
consloe.log(v) // [-1, -2, -3]
```

### 傳入陣列當作初始值

另一種初始化的方式是傳入陣列，陣列的元素會被複製到 Vector 裡面，因此 Vector 和陣列的長度會一樣。

```
const Vector = require('dolphin-node-core')
const initialValues = [1, 2, 3, 4]
const v = new Vector(initialValues)
console.log(v.length === initialValues.length) // true
```

如果傳入多個陣列給 contructor，最後建立的 Vector 會包含所有陣列的元素

```
const Vector = require('dolphin-node-core')
const v = new Vector([1, 2, 3, 4], [5, 6, 7])
console.log(v.length) // 長度等於 7
console.log(v) // [1, 2, 3, 4, 5, 6, 7]
```

如果陣列和數字混在一起傳給 contructor，數字會被當作元素

```
const Vector = require('dolphin-node-core')
const v = new Vector([1, 2, 3], 4, [5, 6, 7])
console.log(v.length) // 長度等於 7
console.log(v) // [1, 2, 3, 4, 5, 6, 7]
```

## add

### 說明

Vector 與數值或是另一個 Vector 相加。

### 參數

`{number|Vector} other` add 可以傳入數值或是 Vector。如果傳入數值，add 會將 Vector 的每一個元素加上傳入的數值。如果傳入 Vector，結果是自身的元素和 Vector 的元素相加，要注意的是傳入 Vector 的長度必須相同，否則會發生錯誤。

## 回傳

`{Vector}` 回傳新的 Vector 。

## 例外

`{TypeError|ParamError}`

## 範例

### 回傳的結果是新向量

下面這段程式將 v1 的每個元素加上 3 ，回傳的結果存在 v2。因為 add 會回傳新的 Vector，所以 v1 和 v2 是不同的物件。

```
const v1 = new Vector([1, 2, 3])
const v2 = v1.add(3)
console.log(v2 === v1) // false
console.log(v1) // [1, 2, 3]
console.log(v2) // [4, 5, 6]
```

如果要在原本的 Vector 做 add 運算，要這樣寫

```
let v = new Vector([1, 2, 3])
v = v.add(3)
console.log(v)
```

### 與 Vector 相加

要將兩個 Vector 相加，可以這樣寫：

```
1  const Vector = require('dolphin-node-core')
2  const v1 = new Vector([1, 2, 3])
3  const v2 = new Vector([4, 5, 6])
4  const v3 = v1.add(v2)
5  console.log(v3) // [5, 7, 9]
6  console.log(v1) // [1, 2, 3]
7  console.log(v2) // [4, 5, 6]
```

前面說過 Vector 可以傳入多個數字作為初始值，所以上面程式的 1~3 行可以改寫成這樣

```
1  const Vector = require('dolphin-node-core')
2  const v = new Vector(1, 2 ,3).add(new Vector(4, 5, 6))
3  console.log(v) // [5, 7, 9]
```

原本傳入陣列，改成傳入多個數字，結果是一樣的。

> 傳入的 `Vector` 長度必須一樣，否則會產生錯誤。

下面的例子是長度為 2 和長度為 3 的 Vector 相加，會產生錯誤

```
const Vector = require('dolphin-node-core')
const v1 = new Vector([1, 2])
const v2 = new Vector([4, 5, 6])
const v3 = v1.add(v2) // 參數other的長度必須相同
```

### 與數字相加

如果呼叫 add 傳入一個數字，結果會是 Vector 每一個元素的加上該數值

```
console.log(new Vector([1, 2, 3]).add(2)) // [3, 4, 5]
```

### 與其他類型的資料相加

如果 add 傳入的參數不是 Vector 或是數值，會發生錯誤。

```
new Vector([1, 2, 3]).add('a') // 參數other的型別必須是數值或向量
```

## append

### 說明

將新的元素附加至 Vector。

### 參數

`{array|number} element` 要加入的元素可以是單一數值或是陣列。

### 回傳

`{Vector}` 回傳新的 Vector 。

### 範例

#### 空向量附加一個數值

```
const Vector = require('dolphin-node-core')
const v = new Vector().append(21)
console.log(v) // [21]
```

#### 加入一個有三個元素的陣列

```
const Vector = require('dolphin-node-core')
const v = new Vector([1]).append([2, 3, 4])
console.log(v) // [1, 2, 3, 4]
```

# clone

## 說明

複製成新 Vector 。

## 參數

無。

## 回傳

`{Vector}` 回傳新的 Vector，內容與自身相同。

## 範例

```
const Vector = require('dolphin-node-core')
const v1 = new Vector([1, 2, 3])
const v2 = v1.clone()
console.log(v2) // [1, 2, 3]
```

# cross

## 說明

向量外積。

## 參數

`{Vector} other` 要進行 cross 運算的 Vector。

## 回傳

`{Vector}` 回傳新的 Vector。

## 例外

{Error} 如果長度不是 3，便會拋出此例外。

## 範例

$\vec{a} \times \vec{a} = 0$

```
const Vector = require('dolphin-node-core')
const a = new Vector([3, 4, 1])
console.log(a.cross(a).equals([0, 0, 0])) // true
```

$\vec{a} \times 0 = 0$

```
const Vector = require('dolphin-node-core')
const a = new Vector([3, 4, 1])
console.log(a.cross([0, 0, 0]).equals([0, 0, 0])) // true
```

### 反交换律 $\vec{a} \times \vec{b} = -\vec{b} \times \vec{a}$

```
const Vector = require('dolphin-node-core')
const a = new Vector([3, 4, 1])
const b = new Vector([3, 7, 5])
const c1 = a.cross(b)
const c2 = b.cross(a).mul(-1)
console.log(c1.equals(c2)) // true
```

### 加法的左分配律 $\vec{a} \times (\vec{b} + \vec{c}) = \vec{a} \times \vec{b} + \vec{a} \times \vec{c}$

```
const Vector = require('dolphin-node-core')
const a = new Vector([3, 4, 1])
const b = new Vector([3, 7, 5])
const c = new Vector([1, 2, 3])
console.log(a.cross(b.add(c)).equals(a.cross(b).add(a.cross(c)))) // true
```

### 加法的右分配律 $(\vec{a} + \vec{b}) \times \vec{c} = \vec{a} \times \vec{c} + \vec{b} \times \vec{c}$

```
const Vector = require('dolphin-node-core')
const a = new Vector([3, 4, 1])
const b = new Vector([3, 7, 5])
const c = new Vector([1, 2, 3])
console.log(a.add(b).cross(c).equals(a.cross(c).add(b.cross(c)))) // true
```

$(\lambda\vec{a}) \times \vec{b} = \lambda(\vec{a} \times \vec{b}) = \vec{a} \times (\lambda\vec{b})$

```
const Vector = require('dolphin-node-core')
const a = new Vector([3, 4, 1])
const b = new Vector([3, 7, 5])
const lambda = 9
const r1 = a.mul(lambda).cross(b)
const r2 = a.cross(b).mul(lambda)
const r3 = a.cross(b.mul(lambda))
console.log(r1.equals(r2)) // true
console.log(r2.equals(r3)) // true
```

$|\vec{a} \times \vec{b}| = |\vec{b} \times \vec{a}|$

```
const Vector = require('dolphin-node-core')
const a = new Vector([3, 4, 1])
const b = new Vector([3, 7, 5])
console.log(a.cross(b).norm() === b.cross(a).norm()) // true
```

$|\vec{a} \times \vec{b}|^2 = |\vec{a}|^2|\vec{b}|^2 - (\vec{a} \cdot \vec{b})^2$

```
const {Vector, Type} = require('dolphin-node-core')
const sqr = (x) => Math.pow(x, 2)
const a = new Vector([3, 4, 1])
const b = new Vector([3, 7, 5])
const r1 = sqr(a.cross(b).norm)
const r2 = sqr(a.norm) * sqr(b.norm) - sqr(a.dot(b))
console.log(Type.equals(r1, r2))
```

# dot

## 說明

向量內積。

## 參數

`{Vector} other` 要進行 dot 運算的 Vector。

## 回傳

`{number}` 數值。

## 例外

{Error} 如果長度不是 3，便會拋出此例外。

## 範例

### 交換律 $\vec{a} \cdot \vec{b} = \vec{b} \cdot \vec{a}$

```
const Vector = require('dolphin-node-core')
const a = new Vector([3, 4, 1])
const b = new Vector([3, 7, 5])
console.log(a.dot(b) === b.dot(a)) // true
```

### 分配律 $\vec{a} \cdot (\vec{b} + \vec{c}) = \vec{a} \cdot \vec{b} + \vec{a} \cdot \vec{c}$

```
const Vector = require('dolphin-node-core')
const a = new Vector([3, 4, 1])
const b = new Vector([3, 7, 5])
const c = new Vector([1, 2, 3])
const d1 = a.dot(b.add(c))
const d2 = a.dot(b) + a.dot(c))
console.log(d1 === d2) // true
```

### 內積是雙線性算子 $\vec{a} \cdot (r\vec{b} + \vec{c}) = r(\vec{a} \cdot \vec{b}) + (\vec{a} \cdot \vec{c})$

```
const Vector = require('dolphin-node-core')
const a = new Vector([3, 4, 1])
const b = new Vector([3, 7, 5])
const c = new Vector([1, 2, 3])
const r = 2
const d1 = a.dot(b.mul(r).add(c))
const d2 = r * a.dot(b) + a.dot(c)
console.log(d1 === d2) // true
```

### 乘以純量時滿足 $(c_1\vec{a}) \cdot (c_2\vec{b}) = (c_1c_2)(\vec{a} \cdot \vec{b})$

```
const Vector = require('dolphin-node-core')
const a = new Vector([3, 4, 1])
const b = new Vector([3, 7, 5])
const c1 = 2
const c2 = 4
const d1 = a.mul(c1).dot(b.mul(c2))
const d2 = c1 * c2 * a.dot(b)
console.log(d1 === d2) // true
```

### 兩個非零向量 $\vec{a}$ 和 $\vec{b}$ 是正交的，$\vec{a} \cdot \vec{b}$ 必為 0

```
const Vector = require('dolphin-node-core')
const a = new Vector([0, -3])
const b = new Vector([3, 0])
console.log(a.dot(b) === 0)
```